2026-6-1 C++红黑树

- 1.红黑树的概念
  - 1.1 红黑树的规则
    - 1.2 红黑树的效率
- 2.红黑树的代码实现
  - 2.1 结点的构造
  - 2.2 插入函数
    - 2.2.1 情况一：变色
    - 2.2.2 情况二：单旋+变色
    - 2.2.3 情况三：双旋+变色
- 3. 检测平衡
- 4. 查找函数
- 5.测试

1.红黑树的概念

红黑树的底层逻辑实际上也是平衡二叉树，他的每个结点都增加了一个存储位来表示结点的颜色，红色或者黑色。对通过任何一条从根到叶子结点的路径上各个结点的颜色进行约束，**红黑树需要确保没有一条路径会比其他路径长出两倍，因而是接近平衡的。**

1.1 红黑树的规则

特点：最长的路径是最短的路径的两倍，平衡的相对松散，长短差距较大，性能上就会优一点。但是AVL树会更均衡一些，旋转代价相对会大一些。

1. 每个结点不是红色就是黑色
2. 根节点必须是黑色的
3. 如果一个结点是红色，则他的两个孩子结点必须是黑色的，这也就以为这任一一条路径不会有连续的红色结点。
4. 对于任一一个结点，从该结点到其所有的NULL结点的简单路径上，均包含颜色数量相同的红色结点。
   上述规则的约束导致了最长路径不超过最短路径的二倍。

1.2 红黑树的效率

假设N是红黑树树中结点数量，h最短路径的长度，那么 , 由此推出
也就是意味着红黑树增删查改最坏也就是走最长路径 ，那么时间复杂度还是

假设 N 是红黑树树中结点数量，h 是最短路径的长度，那么：

$$2^h - 1 \leq N < 2^{2h} - 1$$

由此推出：

$$h \approx \log N$$

这意味着红黑树增删查改最坏情况走最长路径为 (2 \log N)，所以时间复杂度仍然是：

$$O(\log N)$$

红黑树的表达相对 AVL 树要抽象一些，AVL 树通过高度差直观地控制了平衡。红黑树通过 4 条规则的颜色约束，间接实现了近似平衡。两者效率属于同一档次，但插入相同数量的结点时，红黑树的旋转次数更少，因为它对平衡的控制没那么严格。

2.红黑树的代码实现

2.1 结点的构造

1. 枚举值的颜色
2. 默认按照key/value的结构实现，结点是一个三叉链
3. 定义变量pair,_left,_right,_parent,_col（颜色）
4. 构造函数初始化变量

```
enum Color
{
    RED,
    BLACK
};
template < class K, class V>
struct RBTreeNode
{
// 这里更新控制平衡也要加入parent指针
    pair<K, V> _kv;
    RBTreeNode<K, V>* _left;
    RBTreeNode<K, V>* _right;
    RBTreeNode<K, V>* _parent;
    Colour _col;
        RBTreeNode(const pair<K, V>&kv)
            :_kv(kv)
```

```
            , _left(nullptr)
            , _right(nullptr)
            , _parent(nullptr)
            ,_col(RED)
            {}
};
template < class K, class V>
class RBTree
{
    typedef RBTreeNode<K, V> Node;
public:
private:
    Node * _root = nullptr;
};
```

2.2 插入函数

和搜索树类似，区别在于颜色的标记

1. 插入一个值按二叉搜索树规则进行插入，插入后我们只需要观察是否符合红黑树的 4 条规则。
2. 如果是空树插入，新增结点是黑色结点。如果是非空树插入，新增结点必须是红色结点，因为非空树插入新增黑色结点会破坏规则 4（规则 4 很难维护）。
3. 非空树插入后，新增结点必须是红色结点。如果父亲结点是黑色的，则没有违反任何规则，插入结束。
4. 非空树插入后，新增结点必须是红色结点。如果父亲结点是红色的，则违反规则 3。进一步分析，c 是红色，p 为红色，g 必为黑色，这三个颜色都固定了。关键变化取决于 u 的情况，需要根据 u 分为以下几种情况分别处理。

> 说明：下图中假设我们把新增结点标识为 c（cur），c 的父亲标识为 p（parent），p 的父亲标识为 g（grandfather），p 的兄弟标识为 u（uncle）。

> 根节点是黑色
> 如何选择插入红的还是黑的？黑色红色怎么控制？插入红色影响规则三，插入黑色影响规则四。选择插入红色，黑色必然挨打，会破坏每条路径上固有的黑色结点数目。按照搜索树的规则插入，插入到黑色结点下面不用管，插入到红色结点下面则破坏了规则，就调整。意思就是插入红色结点不一定违反规则，插入黑色结点一定违反规则。
> 插入红色，检查父亲，如果父亲的颜色是黑色，就结束了。父亲是红色就要调整了。

2.2.1 情况一：变色

c 为红，p 为红，g 为黑，u 存在且为红，则将 p 和 u 变黑，g 变红，再把 g 当作新的 c，继续往上更新。
因为 p 和 u 都是红色，g 是黑色，把 p 和 u 变黑，左边子树路径各增加一个黑色结点，g 再变红，相当于保持 g 所在子树的黑色结点的数量不变，同时解决了 c 和 p 连续红色结点的问题。需要继续往上更新是因为 g 是红色，如果 g 的父亲还是红色，就需要继续处理；如果 g 的父亲是黑色，则处理结束；如果 g 就是整棵树的根，再把 g 变回黑色。

> 总结：情况 1 只变色，不旋转。所以无论 c 是 p 的左还是右，p 是 g 的左还是右，都是上面的变色处理方式。

2.2.2 情况二：单旋+变色

c为红，p为红，g为黑，u不存在或者u存在且为黑，u不存在，则c一定是新增结点，u存在且为黑，则c一定不是新增，c之前是黑色的，是在c的子树中插入，符合情况1，变色将c从黑色变成红色，更新上来的。
因为p必须变黑，才能解决，连续红色结点的问题，u不存在或者是黑色的，这里单纯的变色无法解决问题，需要旋转+变色。

```
    g
   /
  p  u
 /
c
```

如果p是g的左，c是p的左，那么以g为旋转点进行右单旋，再把p变黑，g变红即可。p变成课这颗树新的根，这样子树黑色结点的数量不变，没有连续的红色结点了，且不需要往上更新，因为p的父亲是黑色还是红色或者空都不违反规则。

```
  g
 / \
u   p
     \
      c
```

如果p是g的右，c是p的右，那么以g为旋转点进行左单旋，再把p变黑，g变红即可。p变成课这颗树新的根，这样子树黑色结点的数量不变，没有连续的红色结点了，且不需要往上更新，因为p的父亲是黑色还是红色或者空都不违反规则。

2.2.3 情况三：双旋+变色

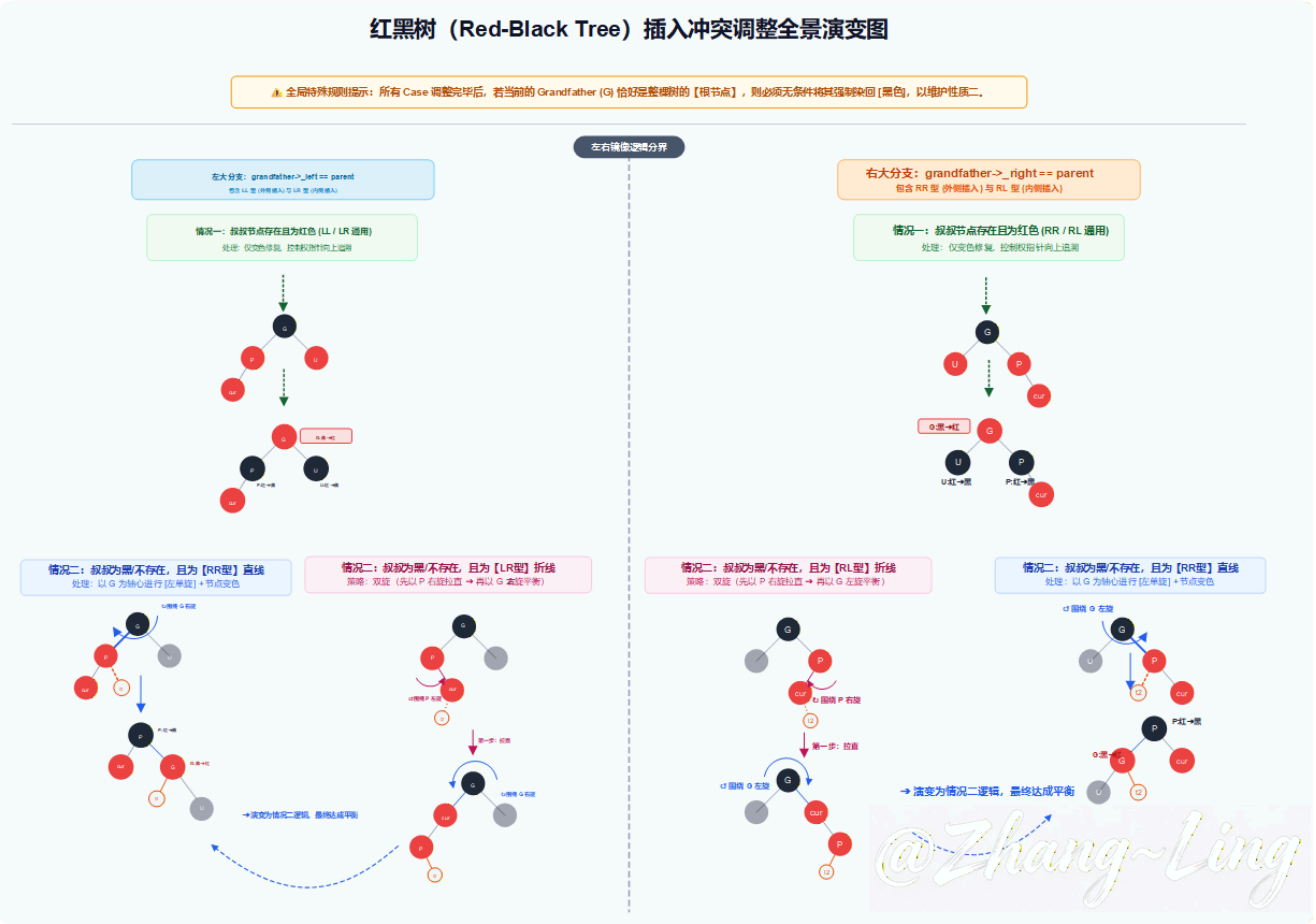
c为红，p为红，g为黑，u不存在或者u存在且为黑，u不存在，则c一定是新增结点，u存在且为黑，则c一定不是新增，c之前是黑色的，是在c的子树中插入，符合情况1，变色将c从黑色变成红色，更新上来的。
p必须变黑，才能解决，连续红色结点的问题，u不存在或者是黑色的，这里单纯的变色无法解决问题，需要旋转+变色。

```
      g
     /  \
    p    u
     \
      c
```

如果p是g的左，c是p的右，那么先以p为旋转点进行左单旋，再以g为旋转点进行右单旋，再把c变黑，g变红即可。c变成课这颗树新的根，这样子树黑色结点的数量不变，没有连续的红色结点了，且不需要往上更新，因为c的父亲是黑色还是红色或者空都不违反规则。

```
  g
 / \
u   p
   /
  c
```

如果p是g的右，c是p的左，那么先以p为旋转点进行右单旋，再以g为旋转点进行左单旋，再把c变黑，g变红即可。c变成课这颗树新的根，这样子树黑色结点的数量不变，没有连续的红色结点了，且不需要往上更新，因为c的父亲是黑色还是红色或者空都不违反规则。
可以总结为如下规则：



代码实现流程：

1. 判断根节点是否为空，为空则创建一个新的结点，并将结点的颜色赋为黑色
2. 使用cur来查询标记插入位置
3. 新增结点，新增结点的颜色给为红色，依据平衡二叉树的规则先将该结点插入到该平衡树中。
4. 依照红黑树的规则对整体的颜色和结构进行调整，判断条件为父亲一直存在且父节点的颜色为红色
5. 定义变量grandfather为parent的parent，我们发现，在已知的几个变量中，只有uncle这个结点是一个随时可能变化的变量，所以这个我们的入手点就是uncle.
6. 如果uncle为你的右节点则分为uncle存在且为红色，则让parent和uncle的值全部变为黑色，再让grandparent的值变为红色，然后向上蔓延；如果uncle不存在或者存在且为黑色则旋转加变色。逻辑梳理为下图：



代码的具体实现：

```
bool Insert(const pair<K, V>& kv)
{
    if (_root == nullptr)
    {
        _root = new Node(kv);
        _root->_col = BLACK;
        return true;
    }

    Node* cur = _root;
    Node* parent = nullptr;

    // 1. 找到插入位置
    while (cur)
    {
        if (kv.first > cur->_kv.first)
        {
            parent = cur;
            cur = cur->_right;
        }
        else if (kv.first < cur->_kv.first)
        {
            parent = cur;
            cur = cur->_left;
        }
        else
        {
            return false;  // 键已存在
        }
    }

    // 2. 插入新节点
    cur = new Node(kv);
```

```
    cur->_col = RED;
    if (parent->_kv.first < kv.first)
    {
        parent->_right = cur;
    }
    else
    {
        parent->_left = cur;
    }
    cur->_parent = parent;

    // 3. 红黑树平衡调整
    while (parent && parent->_col == RED)
    {
        Node* grandparent = parent->_parent;
        if (grandparent == nullptr) break;

        // 情况1：parent 是 grandparent 的左孩子
        if (grandparent->_left == parent)
        {
            Node* uncle = grandparent->_right;

            // 情况1.1：叔叔存在且为红
            if (uncle && uncle->_col == RED)
            {
                parent->_col = BLACK;
                uncle->_col = BLACK;
                grandparent->_col = RED;
                cur = grandparent;
                parent = cur->_parent;
            }
            else
            {
                // 情况1.2：叔叔不存在或为黑
                if (cur == parent->_left)
                {
                    // 右单旋
                    RotateR(grandparent);
                    parent->_col = BLACK;
                    grandparent->_col = RED;
                }
                else
                {
                    // 左右双旋
                    RotateL(parent);
                    RotateR(grandparent);
```

```
                    cur->_col = BLACK;
                    grandparent->_col = RED;
                }
                break;
            }
        }
        // 情况2：parent 是 grandparent 的右孩子
        else
        {
            Node* uncle = grandparent->_left;

            if (uncle && uncle->_col == RED)
            {
                parent->_col = BLACK;
                uncle->_col = BLACK;
                grandparent->_col = RED;
                cur = grandparent;
                parent = cur->_parent;
            }
            else
            {
                if (cur == parent->_right)
                {
                    // 左单旋
                    RotateL(grandparent);
                    parent->_col = BLACK;
                    grandparent->_col = RED;
                }
                else
                {
                    // 右左双旋
                    RotateR(parent);
                    RotateL(grandparent);
                    cur->_col = BLACK;
                    grandparent->_col = RED;
                }
                break;
            }
        }
    }

    _root->_col = BLACK;
    return true;
}
```

3. 检测平衡

这里获取最长路径和最短路径，检查最长路径不超过最短路径的 2 倍是不可行的，因为就算满足这个条件，红黑树也可能颜色不满足规则，当前暂时没出问题，后续继续插入还是会出问题的。所以我们还是去检查 4 点规则，满足这 4 点规则，一定能保证最长路径不超过最短路径的 2 倍。

1. 规则 1 枚举颜色类型，天然实现保证了颜色不是黑色就是红色。
2. 规则 2 直接检查即可
3. 规则 3 前序遍历检查，遇到红色结点查孩子不太方便，因为孩子有两个，且不一定存在，反过来检查父亲的颜色就方便多了
4. 规则 4 前序遍历，遍历过程中用形参记录跟到当前结点的 blackNum（黑色结点数量），前序遍历遇到黑色结点就++blackNum，走到空就计算出了一条路径的黑色结点数量。再任意一条路径黑色结点数量作为参考值，依次进行比较。

代码实现思路：

1. 首先如果根为空返回真
2. 根是红的返回false，这是一个反向检查
3. 定义一个黑色结点数量的参考值赋参考值为0，定义一个leftMost指针遍历整棵二叉树
4. 循环条件为leftMost不为空，循环体中让leftMost一直向左走，并统计该条路径上黑色结点的数量，调用检查函数，将根结点，0，黑色结点的数目作为参数传入。

```
bool IsBalance() {
    if (_root == nullptr) {
        return true;
    }
    if (_root->_col == RED) {
        return false;
    }
    //定义黑色结点的参考值
    int blackRef = 0;
    Node* leftMost = _root;
    while(leftMost){
        if (leftMost->_col == BLACK) {
            blackRef++;
        }
        leftMost = leftMost->_left;
    }
    return Check(_root, 0, blackRef);
}
```

1. 如果传入的结点为空，则判断blackNum的数量与blackNum的数量是否相等，不相等就返回false
2. 判断cur结点的颜色是否为空，为空则返回false
3. 如果cur的为红色，cur的parent不为空，cur的parent的颜色为红色，则说明出现了连续的红色结点返回false,如果cur的颜色为黑色则blackNum的结点++。递归判断知道cur为空且blackNum `==` blackNumRef

```
bool Check(Node* cur, int blackNum, const int blackNumRef) {
    if (cur == nullptr) {
        if (blackNum != blackNumRef) {
            cout << "黑色结点的数量不相等" << endl;
            return false;
        }
        return true;
    }
    if (cur->_col == RED && cur->_parent && cur->_parent->_col == RED) {
        cout << "出现连续的红色结点" << endl;
        return false;
    }
    if (cur->_col == BLACK) {
        blackNum++;
    }
    return Check(cur->_left, blackNum, blackNumRef)
        && Check(cur->_right, blackNum, blackNumRef);
}
```

4. 查找函数

效率为$O(logN)$，按照二叉树的逻辑实现即可：

```
Node* Find(const K& key) {
    Node* cur = _root;
    while (cur) {
        if (key > cur->_kv.first) {
            cur = cur->_right;
        }
        else if (key < cur->_kv.first) {
            cur = cur->_left;
        }
        else {
            return cur;
        }
    }
```

```
    return nullptr;
}
```

5.测试

1. 创建一棵包含特殊情况的二叉树进行测试
2. 一堆随机值进行测试

```
#define _CRT_SECURE_NO_WARNINGS 1
#include<iostream>
#include<vector>
using namespace std;
#include"RBTree.h"

void TestRBTree1()
{
    RBTree<int, int> t;
    // 常规的测试用例
    //int a[] = { 16, 3, 7, 11, 9, 26, 18, 14, 15 };
    // 特殊的带有双旋场景的测试用例
    int a[] = { 4, 2, 6, 1, 3, 5, 15, 7, 16, 14 };
    for (auto e : a)
    {

        t.Insert({ e, e });
        //t.InOrder();
        //cout << "Insert:" << e << "->" << t.IsBalanceTree() << endl;
    }
    t.InOrder();
    cout << t.IsBalance() << endl;
}

// 插入一堆随机值，测试平衡，顺便测试一下高度和性能等
void TestRBTree2()
{
    const int N = 1000000;
    vector<int> v;
    v.reserve(N);
    srand(time(0));
    for (size_t i = 0; i < N; i++)
    {
        v.push_back(rand() + i);
    }
```

```
    size_t begin2 = clock();
    RBTree<int, int> t;
    for (auto e : v)
    {
        t.Insert(make_pair(e, e));
    }
    size_t end2 = clock();
    cout << "Insert:" << end2 - begin2 << endl;
    cout << t.IsBalance() << endl;
    cout << "Height:" << t.Height() << endl;
    cout << "Size:" << t.Size() << endl;

    size_t begin1 = clock();
    // 确定在的值
    /*for (auto e : v)
    {
    t.Find(e);
    }*/
    // 随机值
    for (size_t i = 0; i < N; i++)
    {
        t.Find((rand() + i));
    }
    size_t end1 = clock();

    cout << "Find:" << end1 - begin1 << endl;
}

int main()
{
    TestRBTree2();

    return 0;
}
```